SENTINEL: AI-Powered Phishing Detection System

Complete Technical Architecture & Methodological Reference Guide

1. Executive Summary & Architecture Overview

Sentinel is an enterprise-grade, multi-layered cyber defense application designed to detect credential phishing, document access scams, and social engineering threats in real-time. By combining a **React Vite Frontend Command Center**, a **Node.js Express Backend Gateway**, and an **Advanced Python AI Server**, Sentinel provides automated, high-precision email evaluation without triggering false positives on legitimate services.

Component	Technology Stack	Key Responsibilities
Frontend	React, Vite, Lucide Icons, CSS3	Real-time Dashboard, Threat Intelligence, Activity Feed, Gmail Auth UI
Backend Gateway	Node.js, Express, Axios, Google APIs	OAuth 2.0 Auth, Gmail API Fetching, Parallel Promise.all execution
AI Engine	Python, Flask, Scikit-learn, Joblib	ML Classification, Forward-Chaining Rules, Naive Bayes, Fuzzy Logic

2. Step-by-Step Workflow (How the App Works)

Step 1: User Authentication & Consent

The user connects their account via Google OAuth 2.0. The backend acquires a short-lived access token with read-only Gmail scope (gmail.readonly). No user credentials or passwords are ever stored.

Step 2: Automated Parallel Inbox Ingestion

When the user views the Sentinel dashboard, the frontend initiates a request to /api/emails. The backend queries Gmail's API for the latest 20 email messages and retrieves headers (Subject, From, Date) and snippet content concurrently.

Step 3: Multi-Layer AI Analysis Request

For each email, the backend dispatches a high-speed HTTP request to the Python AI Flask server on http://127.0.0.1:8000/predict. The AI engine processes the full text using 4 independent analysis engines simultaneously.

Step 4: Threat Score Calculation & Override Logic

The AI engine synthesizes predictions: Machine Learning probability, Naive Bayes token frequency, Fuzzy Logic risk metrics, and Rule Engine facts. Whitelist rules check for legitimate service notifications (e.g., Google Security Alerts, ZeroSSL, IPPB) to prevent false positives.

Step 5: Dashboard Visualization & Live Alerts

The frontend receives normalized JSON results containing individual threat reasons, overall risk grade (**HIGH**, **MEDIUM**, **LOW**), and numerical confidence scores (0-100). The UI automatically updates threat counters and flags suspicious messages in red.

3. Detailed Detection Methods & AI Algorithms

Sentinel integrates 5 complementary security engines to guarantee near-zero false negatives and false positives:

Method 1: Forward-Chaining Rule-Based Expert System (Security Rules)

Defined in ai/security_rules.py, this expert system extracts logical facts from raw email text and infers high-level security conditions using forward chaining rules:

- **Fact Extraction:** Identifies suspicious HTTP links, password requests, document access lures ('sign in to view document', 'access document', 'confirm access'), urgent threats ('account blocked'), and payment requests.
- **Forward Chaining Rule:** IF (suspicious_link OR http_link) AND (asks_password OR document_lure) THEN fact = 'credential_phishing'

Method 2: Supervised Machine Learning (Logistic Regression)

Implemented in ai/train_model.py and deployed in ai/predict.py, the ML model classifies text into phishing or safe:

- **Feature Extraction:** Uses TF-IDF Vectorizer with 1-2 word n-grams (ngram_range=(1,2)) and max_features=5000.
- **Model Parameterization:** Trained with class_weight='balanced' and regularization parameter C=2.0 to heavily penalize misclassification of credential phishing.
- **Probability Output:** P(Phishing|Text) = 1 / (1 + e^-(w·x + b))

Method 3: Naive Bayes Risk Calculation

Implemented in ai/bayes_risk.py, Naive Bayes computes word likelihoods across known scam corpus vs safe corpus:

- Calculates Bayes risk percentage: BayesRisk = P(Spam|Words) = $\prod$ P(Word_i|Spam) / [$\prod$ P(Word_i|Spam) + $\prod$ P(Word_i|Safe)]

Method 4: Fuzzy Logic Risk Engine

Implemented in ai/fuzzy_risk.py, Fuzzy Logic handles ambiguity in email tone, link count, and domain trust scores:

- **Inputs:** Link frequency, Urgency level, Domain reputation.
- **Fuzzy Membership Output:** Assigns qualitative risk levels (LOW, MEDIUM, HIGH) and numerical fuzzy score (0-100).

Method 5: Whitelist & Trusted Service Pattern Engine

Located in ai/ai_server.py, this engine scans emails for known trusted transactional patterns (Google Account Security Alerts, India Post Payment Bank, ZeroSSL expiry notices) to ensure legitimate operational emails are never classified as phishing.

4. Final Decision Matrix & Risk Thresholds

Final Risk Grade	Trigger Condition / Evidence Requirement	UI Action
HIGH RISK	Rule-based credential_phishing fact TRUE **OR** ML Prob ≥ 85% **OR** (ML Phishing & Rule Score ≥ 30)	Red alert badge, flagged in Threat Intelligence
MEDIUM RISK	ML Phishing with 50-84% confidence **OR** Rule score 30-55 **OR** Bayes Risk ≥ 50%	Amber review badge
LOW RISK	Trusted service whitelist match **OR** Safe ML prediction with no critical rule violations	Green secure badge